CropScan V4 — DINOv2 + ensemble + stronger synthesis

Target: push PlantDoc accuracy from ~68% (v3_new) to 75%+.

Changes vs v3_new:

1. **DINOv2-S/14** as primary backbone — self-supervised features generalize much better than ImageNet-supervised CNNs across the lab→field domain shift. Linear probe (frozen backbone) + small MLP head.
2. **EfficientNetV2-S** kept as secondary, trained from scratch with same recipe.
3. **Stronger synthesis: 25K images** (vs 8.8K) with color jitter on the leaf, random shadow gradient, larger rotations, 3 backgrounds per leaf.
4. **EMA weights** during training — eval against the EMA copy.
5. **Multi-resolution eval**: train at 224, eval at 280 with 10-crop TTA.
6. **Ensemble at inference**: average softmax from DINOv2 + EffNet (both calibrated).
7. **Lower abstention precision target**: 0.80 instead of 0.90, so thresholds actually exist on PlantDoc.

Required data on disk (paths in cell 4):

- PlantVillage 38 class folders
- PlantDoc with `train/` and `test/` subfolders
- The synthetic-field backgrounds from v3 (reused; if missing, regenerate)

Outputs (in `artifacts/v4/`):

- `dinov2_vits14_cropscan_v4.pth` + `efficientnet_v2_s_cropscan_v4.pth`
- matching `*_bundle.pt` files (state_dict + temperature + thresholds + history)
- `training_report.json`, `labels.json`, preview PNGs

Expected runtime on A6000 ~4-5 hours: synth ~30 min, DINOv2 ~75 min, EffNet ~90 min.

In [2]:
```
import copy, csv, json, math, os, random, subprocess, sys
from collections import Counter, defaultdict
from dataclasses import asdict, dataclass
from io import BytesIO
from pathlib import Path

import numpy as np
import torch
import torch.nn as nn
from PIL import Image, ImageFilter
from sklearn.metrics import f1_score
from torch.utils.data import DataLoader, Dataset, WeightedRandomSampler
from torchvision import models, transforms
```

```
print(f'Python:  {sys.version.split()[0]}')
print(f'PyTorch: {torch.__version__}')
print(f'CUDA:    {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU:     {torch.cuda.get_device_name(0)} ({torch.cuda.get_device_proper
```

```
Python:  3.12.3
PyTorch: 2.6.0+cu124
CUDA:    True
GPU:     NVIDIA RTX A6000 (50.9 GB)
```

In [3]:
```
SEED = 434
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED); torch.cuda.manual
torch.backends.cudnn.benchmark = True
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')
```

```
Device: cuda
```

Paths and hyperparameters

In [4]:
```
PLANTVILLAGE_DIR = Path('color')
PLANTDOC_DIR     = Path('data/plantdoc')

ARTIFACT_DIR  = Path('artifacts/v4')
SYNTHETIC_DIR = Path('data/synthetic_field')
SYNTH_LEAVES_DIR = SYNTHETIC_DIR / 'leaves_v4'
SYNTH_INDEX = SYNTHETIC_DIR / 'synth_index_v4.json'
BG_DIR = SYNTHETIC_DIR / 'backgrounds'
for d in [ARTIFACT_DIR, SYNTHETIC_DIR, SYNTH_LEAVES_DIR, BG_DIR]:
    d.mkdir(parents=True, exist_ok=True)

@dataclass
class Config:
    image_size_train: int = 224
    image_size_eval: int  = 280
    batch_size: int = 32
    epochs: int = 25
    warmup_epochs: int = 3
    learning_rate: float = 5e-4
    dinov2_head_lr: float = 1e-3
    weight_decay: float = 1e-4
    num_workers: int = 4
    label_smoothing: float = 0.1

    plantvillage_weight: float = 1.0
    synthetic_weight:    float = 2.5
    plantdoc_weight:     float = 6.0

    mixup_alpha: float = 0.2
    cutmix_alpha: float = 1.0
    mix_prob: float = 0.5

    num_synth_per_leaf: int = 3
```

```
    max_synth_total: int = 25000
    n_background_patches: int = 2000

    ema_decay: float = 0.999

    target_confident_precision: float = 0.80

CFG = Config()
print(json.dumps(asdict(CFG), indent=2))
```

```
{
  "image_size_train": 224,
  "image_size_eval": 280,
  "batch_size": 32,
  "epochs": 25,
  "warmup_epochs": 3,
  "learning_rate": 0.0005,
  "dinov2_head_lr": 0.001,
  "weight_decay": 0.0001,
  "num_workers": 4,
  "label_smoothing": 0.1,
  "plantvillage_weight": 1.0,
  "synthetic_weight": 2.5,
  "plantdoc_weight": 6.0,
  "mixup_alpha": 0.2,
  "cutmix_alpha": 1.0,
  "mix_prob": 0.5,
  "num_synth_per_leaf": 3,
  "max_synth_total": 25000,
  "n_background_patches": 2000,
  "ema_decay": 0.999,
  "target_confident_precision": 0.8
}
```

In [5]:
```
CLASS_NAMES = [
    'Apple___Apple_scab', 'Apple___Black_rot', 'Apple___Cedar_apple_rust', 'Apple__
    'Blueberry___healthy',
    'Cherry_(including_sour)___Powdery_mildew', 'Cherry_(including_sour)___healthy'
    'Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot', 'Corn_(maize)___Common_ru
    'Corn_(maize)___Northern_Leaf_Blight', 'Corn_(maize)___healthy',
    'Grape___Black_rot', 'Grape___Esca_(Black_Measles)',
    'Grape___Leaf_blight_(Isariopsis_Leaf_Spot)', 'Grape___healthy',
    'Orange___Haunglongbing_(Citrus_greening)',
    'Peach___Bacterial_spot', 'Peach___healthy',
    'Pepper__bell___Bacterial_spot', 'Pepper__bell___healthy',
    'Potato___Early_blight', 'Potato___Late_blight', 'Potato___healthy',
    'Raspberry___healthy',
    'Soybean___healthy',
    'Squash___Powdery_mildew',
    'Strawberry___Leaf_scorch', 'Strawberry___healthy',
    'Tomato_Bacterial_spot', 'Tomato_Early_blight', 'Tomato_Late_blight',
    'Tomato_Leaf_Mold', 'Tomato_Septoria_leaf_spot',
    'Tomato_Spider_mites_Two_spotted_spider_mite', 'Tomato__Target_Spot',
    'Tomato__Tomato_YellowLeaf__Curl_Virus', 'Tomato__Tomato_mosaic_virus',
    'Tomato_healthy',
]
```

```
CLASS_TO_IDX = {n: i for i, n in enumerate(CLASS_NAMES)}
NUM_CLASSES = len(CLASS_NAMES)
IMG_EXTS = {'.jpg', '.jpeg', '.png', '.webp'}
assert NUM_CLASSES == 38
print(f'{NUM_CLASSES} classes')
```

```
38 classes
```

In [6]:
```
def count_images(root: Path) -> int:
    if not root.exists():
        return 0
    return sum(1 for p in root.rglob('*') if p.suffix.lower() in IMG_EXTS)

n_pv = count_images(PLANTVILLAGE_DIR)
n_pd = count_images(PLANTDOC_DIR)
print(f'[{"OK " if n_pv else "MISS"}] PlantVillage  {n_pv:>7d} images at {PLANTVILL
print(f'[{"OK " if n_pd else "MISS"}] PlantDoc      {n_pd:>7d} images at {PLANTDOC_
n_bg = len(list(BG_DIR.glob('bg_*.jpg')))
print(f'[{"OK " if n_bg else "MISS"}] Backgrounds   {n_bg:>7d} cached at {BG_DIR}')
assert n_pv > 0, f'PlantVillage missing at {PLANTVILLAGE_DIR}'
assert n_pd > 0, f'PlantDoc missing at {PLANTDOC_DIR}'
```

```
[OK ] PlantVillage    54305 images at color
[OK ] PlantDoc         2579 images at data/plantdoc
[OK ] Backgrounds      2000 cached at data/synthetic_field/backgrounds
```

Class mappings (PlantVillage aliases + PlantDoc mapping)

In [7]:
```
PV_FOLDER_ALIASES = {
    'Pepper,_bell___Bacterial_spot':                'Pepper__bell___Bacterial_spot
    'Pepper,_bell___healthy':                       'Pepper__bell___healthy',
    'Tomato___Bacterial_spot':                      'Tomato_Bacterial_spot',
    'Tomato___Early_blight':                        'Tomato_Early_blight',
    'Tomato___Late_blight':                         'Tomato_Late_blight',
    'Tomato___Leaf_Mold':                           'Tomato_Leaf_Mold',
    'Tomato___Septoria_leaf_spot':                  'Tomato_Septoria_leaf_spot',
    'Tomato___Spider_mites Two-spotted_spider_mite': 'Tomato_Spider_mites_Two_spott
    'Tomato___Target_Spot':                         'Tomato__Target_Spot',
    'Tomato___Tomato_Yellow_Leaf_Curl_Virus':       'Tomato__Tomato_YellowLeaf__Cu
    'Tomato___Tomato_mosaic_virus':                 'Tomato__Tomato_mosaic_virus',
    'Tomato___healthy':                             'Tomato_healthy',
}
assert set(PV_FOLDER_ALIASES.values()).issubset(CLASS_TO_IDX)

PLANTDOC_TO_CROPSCAN = {
    'Apple Scab Leaf': 'Apple___Apple_scab',
    'Apple rust leaf': 'Apple___Cedar_apple_rust',
    'Bell_pepper leaf spot': 'Pepper__bell___Bacterial_spot',
    'Corn Gray leaf spot': 'Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot',
    'Corn leaf blight': 'Corn_(maize)___Northern_Leaf_Blight',
    'Corn rust leaf': 'Corn_(maize)___Common_rust_',
    'Potato leaf early blight': 'Potato___Early_blight',
    'Potato leaf late blight': 'Potato___Late_blight',
    'Squash Powdery mildew leaf': 'Squash___Powdery_mildew',
```

```
    'Tomato Early blight leaf': 'Tomato_Early_blight',
    'Tomato Septoria leaf spot': 'Tomato_Septoria_leaf_spot',
    'Tomato leaf bacterial spot': 'Tomato_Bacterial_spot',
    'Tomato leaf late blight': 'Tomato_Late_blight',
    'Tomato leaf mosaic virus': 'Tomato__Tomato_mosaic_virus',
    'Tomato leaf yellow virus': 'Tomato__Tomato_YellowLeaf__Curl_Virus',
    'Tomato mold leaf': 'Tomato_Leaf_Mold',
    'Tomato two spotted spider mites leaf': 'Tomato_Spider_mites_Two_spotted_spider
    'grape leaf black rot': 'Grape___Black_rot',
}
assert set(PLANTDOC_TO_CROPSCAN.values()).issubset(CLASS_TO_IDX)
```

Collect samples

In [8]:
```
@dataclass
class Sample:
    path: Path
    label: int
    source: str

def collect_folder(root: Path, mapping: dict, source: str):
    if not root.exists():
        return []
    out, skipped = [], []
    for folder in sorted(p for p in root.iterdir() if p.is_dir()):
        mapped = mapping.get(folder.name)
        if mapped is None:
            skipped.append(folder.name)
            continue
        label = CLASS_TO_IDX[mapped]
        for img in folder.rglob('*'):
            if img.suffix.lower() in IMG_EXTS:
                out.append(Sample(img, label, source))
    if skipped:
        print(f'  {root}: skipped {len(skipped)} unmapped folders')
    return out

pv_map = {n: n for n in CLASS_NAMES}
pv_map.update(PV_FOLDER_ALIASES)
pv_samples = collect_folder(PLANTVILLAGE_DIR, pv_map, 'plantvillage')
print(f'PlantVillage: {len(pv_samples)} images, {len({s.label for s in pv_samples})}
```

```
  color: skipped 1 unmapped folders
PlantVillage: 54305 images, 38/38 classes
```

In [9]:
```
pd_samples = []
for sub in ['train', 'test', '']:
    pd_samples.extend(collect_folder(PLANTDOC_DIR / sub if sub else PLANTDOC_DIR, P
seen = set()
pd_unique = []
for s in pd_samples:
    k = str(s.path.resolve())
    if k not in seen:
        seen.add(k)
        pd_unique.append(s)
```

```
pd_samples = pd_unique
print(f'PlantDoc: {len(pd_samples)} images, {len({s.label for s in pd_samples})}/{N
```

```
  data/plantdoc/train: skipped 10 unmapped folders
  data/plantdoc/test: skipped 10 unmapped folders
  data/plantdoc: skipped 3 unmapped folders
PlantDoc: 1728 images, 18/38 classes
```

Stratified splits

In [10]:
```
def stratified_split(samples, val_ratio=0.1, test_ratio=0.1, seed=SEED):
    grouped = defaultdict(list)
    for s in samples:
        grouped[s.label].append(s)
    train, val, test = [], [], []
    rng = random.Random(seed)
    for arr in grouped.values():
        rng.shuffle(arr)
        n = len(arr)
        nt = max(1, int(n * test_ratio)) if n >= 10 else max(0, int(n * test_ratio)
        nv = max(1, int(n * val_ratio))  if n >= 10 else max(0, int(n * val_ratio))
        test.extend(arr[:nt])
        val.extend(arr[nt:nt+nv])
        train.extend(arr[nt+nv:])
    rng.shuffle(train); rng.shuffle(val); rng.shuffle(test)
    return train, val, test

pv_train, pv_val, pv_test = stratified_split(pv_samples, 0.10, 0.10)
pd_train, pd_val, pd_test = stratified_split(pd_samples, 0.15, 0.15)
print(f'pv  train/val/test = {len(pv_train)}/{len(pv_val)}/{len(pv_test)}')
print(f'pd  train/val/test = {len(pd_train)}/{len(pd_val)}/{len(pd_test)}')
```

```
pv  train/val/test = 43471/5417/5417
pd  train/val/test = 1228/250/250
```

Leaf segmentation (corner-based)

In [11]:
```python
def segment_leaf(pil_img, dist_threshold=28, min_fg_ratio=0.05):
    arr = np.asarray(pil_img.convert('RGB'))
    h, w = arr.shape[:2]
    cs = max(8, min(h, w) // 12)
    corners = np.concatenate([
        arr[:cs, :cs].reshape(-1, 3),
        arr[:cs, -cs:].reshape(-1, 3),
        arr[-cs:, :cs].reshape(-1, 3),
        arr[-cs:, -cs:].reshape(-1, 3),
    ])
    bg = np.median(corners, axis=0).astype(np.float32)
    diff = arr.astype(np.float32) - bg
    dist = np.sqrt((diff ** 2).sum(axis=2))
    mask = (dist > dist_threshold).astype(np.uint8) * 255
    if mask.mean() / 255.0 < min_fg_ratio:
        return None
    return Image.fromarray(np.dstack([arr, mask]).astype(np.uint8), 'RGBA')
```

Background pool (reused from v3)

In [12]:
```python
def build_background_pool(source_samples, n_target, patch_size=384):
    existing = sorted(BG_DIR.glob('bg_*.jpg'))
    if len(existing) >= n_target:
        print(f'  cached background pool: {len(existing)} patches')
        return [str(p) for p in existing[:n_target]]
    rng = random.Random(SEED + 11)
    sources = list(source_samples)
    rng.shuffle(sources)
    saved = list(existing)
    i = len(saved)
    for sample in sources:
        if i >= n_target:
            break
        try:
            img = Image.open(sample.path).convert('RGB')
            w, h = img.size
            if min(w, h) < patch_size:
                scale = patch_size / min(w, h) * 1.1
                img = img.resize((int(w * scale), int(h * scale)))
                w, h = img.size
            for _ in range(2):
                if i >= n_target:
                    break
                x = rng.randint(0, w - patch_size)
                y = rng.randint(0, h - patch_size)
                crop = img.crop((x, y, x + patch_size, y + patch_size))
                crop = crop.filter(ImageFilter.GaussianBlur(radius=3.5))
                out = BG_DIR / f'bg_{i:05d}.jpg'
                crop.save(out, quality=85)
                saved.append(out)
                i += 1
        except Exception:
            continue
    print(f'  saved {len(saved)} background patches')
```

```
    return [str(p) for p in saved]

bg_pool = build_background_pool(pd_samples, CFG.n_background_patches)
print(f'Background pool: {len(bg_pool)}')
```

```
 cached background pool: 2000 patches
Background pool: 2000
```

Stronger synthesis — 25K with color jitter, shadows, larger rotations

Augments applied per pasted leaf (before pasting):

- random brightness ±30%, contrast ±15%, per-channel hue shift
- random one-sided shadow gradient (horizontal or vertical)
- rotation ±45 degrees
- 3 backgrounds per source leaf instead of 2

In [13]:
```
def augment_leaf(leaf_rgba, rng):
    arr = np.asarray(leaf_rgba).astype(np.float32)
    rgb = arr[:, :, :3]
    alpha = arr[:, :, 3]

    brightness = rng.uniform(0.7, 1.3)
    rgb = rgb * brightness

    contrast = rng.uniform(0.85, 1.15)
    rgb = (rgb - 128.0) * contrast + 128.0

    rgb[:, :, 0] += rng.uniform(-15, 15)
    rgb[:, :, 1] += rng.uniform(-10, 10)
    rgb[:, :, 2] += rng.uniform(-15, 15)

    if rng.random() < 0.45:
        h, w = alpha.shape
        low = rng.uniform(0.35, 0.85)
        if rng.random() < 0.5:
            grad = np.linspace(low, 1.0, w)
            if rng.random() < 0.5:
                grad = grad[::-1]
            grad = np.tile(grad, (h, 1))
        else:
            grad = np.linspace(low, 1.0, h)
            if rng.random() < 0.5:
                grad = grad[::-1]
            grad = np.tile(grad.reshape(-1, 1), (1, w))
        rgb = rgb * grad[:, :, None]

    rgb = np.clip(rgb, 0, 255).astype(np.uint8)
    out = np.dstack([rgb, alpha.astype(np.uint8)])
    return Image.fromarray(out, 'RGBA')
```

In [14]:

```
def synthesize(pv_train_, backgrounds, num_per_leaf, max_total):
    if SYNTH_INDEX.exists():
        cached = json.load(open(SYNTH_INDEX))
        recovered = [Sample(Path(e['path']), e['label'], 'synthetic') for e in cach
        if len(recovered) >= 0.9 * max_total:
            print(f'  cached synthetic: {len(recovered)}')
            return recovered
    if not backgrounds:
        print('  no backgrounds available, skipping')
        return []
    rng = random.Random(SEED + 17)
    by_class = defaultdict(list)
    for s in pv_train_:
        by_class[s.label].append(s)
    leaves_per_class = max(4, (max_total // max(1, len(by_class))) // num_per_leaf)
    print(f'  target: {leaves_per_class} leaves x {num_per_leaf} bgs x {len(by_clas
    out, total = [], 0
    for label, items in by_class.items():
        rng.shuffle(items)
        used = 0
        for source in items:
            if used >= leaves_per_class or total >= max_total:
                break
            try:
                img = Image.open(source.path).convert('RGB')
                cutout = segment_leaf(img)
                if cutout is None:
                    continue
                alpha = np.asarray(cutout)[:, :, 3]
                ys, xs = np.where(alpha > 32)
                if ys.size == 0:
                    continue
                cutout = cutout.crop((int(xs.min()), int(ys.min()), int(xs.max()) +
            except Exception:
                continue
            for variant in range(num_per_leaf):
                if total >= max_total:
                    break
                try:
                    leaf = augment_leaf(cutout, rng)
                    bg = Image.open(rng.choice(backgrounds)).convert('RGB').copy()
                    bw, bh = bg.size
                    target_side = int(min(bw, bh) * rng.uniform(0.50, 0.95))
                    lw, lh = leaf.size
                    scale = target_side / max(lw, lh)
                    leaf = leaf.resize((max(8, int(lw * scale)), max(8, int(lh * sc
                    leaf = leaf.rotate(rng.uniform(-45, 45), expand=True, resample=
                    lw, lh = leaf.size
                    if lw >= bw or lh >= bh:
                        continue
                    bg.paste(leaf, (rng.randint(0, bw - lw), rng.randint(0, bh - lh
                    out_path = SYNTH_LEAVES_DIR / f'synth_c{label:02d}_{source.path
                    bg.save(out_path, quality=85)
                    out.append(Sample(out_path, label, 'synthetic'))
                    total += 1
```

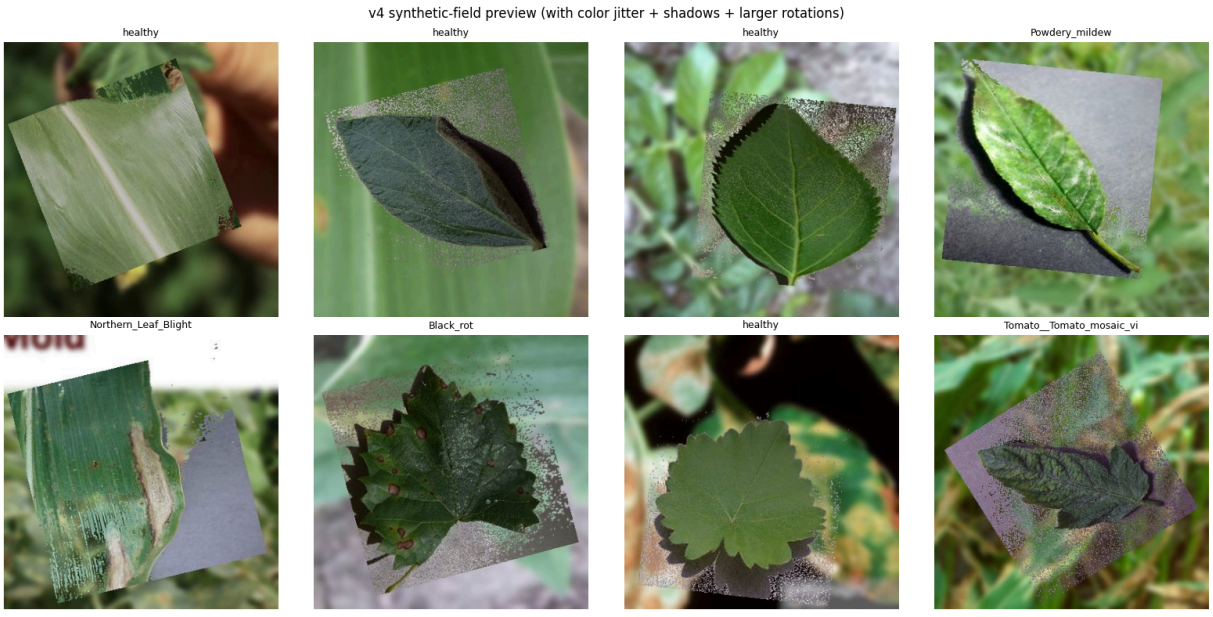
```
                except Exception:
                    continue
            used += 1
        if total and total % 2000 < num_per_leaf:
            print(f'    progress: {total}/{max_total}')
    json.dump([{'path': str(s.path), 'label': s.label} for s in out], open(SYNTH_IN
    print(f'  generated {len(out)} synthetic samples')
    return out

synth_samples = synthesize(pv_train, bg_pool, CFG.num_synth_per_leaf, CFG.max_synth
print(f'Synthetic-field: {len(synth_samples)}')
```

```
  target: 219 leaves x 3 bgs x 38 classes
    progress: 14002/25000
  generated 15925 synthetic samples
Synthetic-field: 15925
```

In [15]:

```
import matplotlib.pyplot as plt

if synth_samples:
    rng = random.Random(SEED + 23)
    picks = rng.sample(synth_samples, min(8, len(synth_samples)))
    fig, axes = plt.subplots(2, 4, figsize=(16, 8))
    for ax, s in zip(axes.flat, picks):
        ax.imshow(Image.open(s.path))
        ax.set_title(CLASS_NAMES[s.label].split('___')[-1][:24], fontsize=9)
        ax.axis('off')
    plt.suptitle('v4 synthetic-field preview (with color jitter + shadows + larger
    plt.tight_layout()
    plt.savefig(ARTIFACT_DIR / 'synth_preview.png', dpi=100, bbox_inches='tight')
    plt.show()
```

```
/opt/jupyterhub/jupyter_env/lib/python3.12/site-packages/matplotlib/projections/__in
it__.py:63: UserWarning: Unable to import Axes3D. This may be due to multiple versio
ns of Matplotlib being installed (e.g. as a system package and as a pip package). As
a result, the 3D projection is not available.
  warnings.warn("Unable to import Axes3D. This may be due to multiple versions of "
```



Training pool + sampler

In [16]:
```
train_samples = pv_train + synth_samples + pd_train
val_samples   = pv_val + pd_val
test_sets = {'plantvillage_test': pv_test, 'plantdoc_test': pd_test}
calibration_samples = pd_val if len(pd_val) >= 100 else val_samples

print(f'TRAIN total: {len(train_samples)}  (pv={len(pv_train)}  synth={len(synth_sa
print(f'VAL   total: {len(val_samples)}')
print(f'CALIB total: {len(calibration_samples)}')
for n, s in test_sets.items():
    print(f'TEST  {n:20s} {len(s)}')
```

```
TRAIN total: 60624  (pv=43471  synth=15925  pd=1228)
VAL   total: 5667
CALIB total: 250
TEST  plantvillage_test    5417
TEST  plantdoc_test        250
```

In [17]:
```
SOURCE_WEIGHTS = {
    'plantvillage': CFG.plantvillage_weight,
    'synthetic':    CFG.synthetic_weight,
    'plantdoc':     CFG.plantdoc_weight,
}

def weighted_sampler(samples):
    label_counts = Counter(s.label for s in samples)
    weights = [SOURCE_WEIGHTS.get(s.source, 1.0) / max(1, label_counts[s.label]) fo
    return WeightedRandomSampler(weights, num_samples=len(weights), replacement=Tru
```

Transforms — train at 224, eval at 280

In [18]:
```
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

train_transform = transforms.Compose([
    transforms.RandomResizedCrop(CFG.image_size_train, scale=(0.5, 1.0), ratio=(0.7
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(p=0.15),
    transforms.RandAugment(num_ops=2, magnitude=9),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
    transforms.RandomErasing(p=0.25, scale=(0.02, 0.2)),
])
eval_transform = transforms.Compose([
    transforms.Resize(CFG.image_size_eval + 32),
    transforms.CenterCrop(CFG.image_size_eval),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

class LeafDataset(Dataset):
    def __init__(self, samples, tfm):
```

```
        self.samples = samples
        self.tfm = tfm
    def __len__(self):
        return len(self.samples)
    def __getitem__(self, i):
        s = self.samples[i]
        try:
            img = Image.open(s.path).convert('RGB')
        except Exception:
            img = Image.new('RGB', (CFG.image_size_train, CFG.image_size_train))
        return self.tfm(img), s.label

train_loader = DataLoader(LeafDataset(train_samples, train_transform), batch_size=C
                          sampler=weighted_sampler(train_samples), num_workers=CFG.
                          pin_memory=torch.cuda.is_available(), drop_last=True,
                          persistent_workers=CFG.num_workers > 0)
val_loader = DataLoader(LeafDataset(val_samples, eval_transform), batch_size=CFG.ba
                        shuffle=False, num_workers=CFG.num_workers,
                        pin_memory=torch.cuda.is_available(),
                        persistent_workers=CFG.num_workers > 0)
calibration_loader = DataLoader(LeafDataset(calibration_samples, eval_transform), b
                                shuffle=False, num_workers=CFG.num_workers,
                                pin_memory=torch.cuda.is_available())
test_loaders = {n: DataLoader(LeafDataset(s, eval_transform), batch_size=CFG.batch_
                              shuffle=False, num_workers=CFG.num_workers,
                              pin_memory=torch.cuda.is_available())
                for n, s in test_sets.items() if s}

x, y = next(iter(train_loader))
print(f'Batch: {tuple(x.shape)}  labels [{y.min().item()}, {y.max().item()}]  train
```

```
Batch: (32, 3, 224, 224)  labels [5, 37]  train batches/epoch=1894
```

MixUp + CutMix

In [19]:
```python
def mixup(x, y, alpha):
    lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
    idx = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1 - lam) * x[idx], y, y[idx], lam

def cutmix(x, y, alpha):
    lam = float(np.random.beta(alpha, alpha)) if alpha > 0 else 1.0
    idx = torch.randperm(x.size(0), device=x.device)
    _, _, H, W = x.size()
    cr = math.sqrt(1 - lam)
    cw, ch = int(W * cr), int(H * cr)
    cx, cy = int(np.random.randint(W)), int(np.random.randint(H))
    x1, y1 = max(cx - cw // 2, 0), max(cy - ch // 2, 0)
    x2, y2 = min(cx + cw // 2, W), min(cy + ch // 2, H)
    x = x.clone()
    x[:, :, y1:y2, x1:x2] = x[idx, :, y1:y2, x1:x2]
    return x, y, y[idx], 1 - ((x2 - x1) * (y2 - y1) / (W * H))

def mix_loss(crit, logits, ya, yb, lam):
    return lam * crit(logits, ya) + (1 - lam) * crit(logits, yb)
```

Models: DINOv2 (frozen backbone + MLP head) and EfficientNetV2-S

In [20]:
```python
class DINOv2Classifier(nn.Module):
    """DINOv2 ViT-S/14 backbone (frozen) + small MLP head over the CLS token."""
    def __init__(self, num_classes: int, embed_dim: int = 384, hidden: int = 512, d
        super().__init__()
        self.backbone = torch.hub.load('facebookresearch/dinov2', 'dinov2_vits14',
        for p in self.backbone.parameters():
            p.requires_grad = False
        self.backbone.eval()
        self.head = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, hidden),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, num_classes),
        )
    def train(self, mode: bool = True):
        super().train(mode)
        self.backbone.eval()
        return self
    def forward(self, x):
        features = self.backbone(x)
        return self.head(features)

def build_model(name):
    if name == 'dinov2_vits14':
        return DINOv2Classifier(NUM_CLASSES)
    if name == 'efficientnet_v2_s':
        m = models.efficientnet_v2_s(weights=models.EfficientNet_V2_S_Weights.IMAGE
        m.classifier = nn.Sequential(nn.Dropout(0.3, inplace=True),
```

```
                                        nn.Linear(m.classifier[1].in_features, NUM_CL
        return m
    raise ValueError(name)

for n in ['dinov2_vits14', 'efficientnet_v2_s']:
    m = build_model(n)
    total = sum(p.numel() for p in m.parameters()) / 1e6
    trainable = sum(p.numel() for p in m.parameters() if p.requires_grad) / 1e6
    print(f'{n:25s} total={total:.1f}M  trainable={trainable:.2f}M')
    del m
```

```
Downloading: "https://github.com/facebookresearch/dinov2/zipball/main" to /home/span
t/.cache/torch/hub/main.zip
/home/spant/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/swiglu_ffn.p
y:51: UserWarning: xFormers is not available (SwiGLU)
  warnings.warn("xFormers is not available (SwiGLU)")
/home/spant/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/attention.p
y:33: UserWarning: xFormers is not available (Attention)
  warnings.warn("xFormers is not available (Attention)")
/home/spant/.cache/torch/hub/facebookresearch_dinov2_main/dinov2/layers/block.py:40:
UserWarning: xFormers is not available (Block)
  warnings.warn("xFormers is not available (Block)")
Downloading: "https://dl.fbaipublicfiles.com/dinov2/dinov2_vits14/dinov2_vits14_pret
rain.pth" to /home/spant/.cache/torch/hub/checkpoints/dinov2_vits14_pretrain.pth
100%|██████████| 84.2M/84.2M [00:00<00:00, 110MB/s]
dinov2_vits14             total=22.3M  trainable=0.22M
efficientnet_v2_s         total=20.2M  trainable=20.23M
```

EMA model wrapper

In [21]:

```
class EMA:
    def __init__(self, model, decay=0.999):
        self.decay = decay
        self.shadow = {k: v.detach().clone() for k, v in model.state_dict().items()
    @torch.no_grad()
    def update(self, model):
        for k, v in model.state_dict().items():
            if v.dtype.is_floating_point:
                self.shadow[k].mul_(self.decay).add_(v.detach(), alpha=1 - self.dec
            else:
                self.shadow[k].copy_(v.detach())
    def apply_to(self, model):
        model.load_state_dict(self.shadow)
```

Scheduler + eval helpers + training loop

In [22]:

```
def warmup_cosine(optimizer, warmup_steps, total_steps, min_ratio=0.01):
    def fn(step):
        if step < warmup_steps:
            return (step + 1) / max(1, warmup_steps)
        prog = (step - warmup_steps) / max(1, total_steps - warmup_steps)
        return max(min_ratio, 0.5 * (1 + math.cos(math.pi * min(1.0, prog))))
    return torch.optim.lr_scheduler.LambdaLR(optimizer, fn)
```

```
@torch.no_grad()
def collect_logits(model, loader):
    model.eval()
    L, Y = [], []
    for x, y in loader:
        L.append(model(x.to(device, non_blocking=True)).cpu())
        Y.append(y)
    return torch.cat(L), torch.cat(Y)

def evaluate_logits(logits, labels, T=1.0):
    p = torch.softmax(logits / T, dim=1)
    pred = p.argmax(1)
    return {
        'accuracy': float((pred == labels).float().mean()),
        'macro_f1': float(f1_score(labels.numpy(), pred.numpy(), average='macro', z
    }
```

In [23]:

```
def train(name, head_lr=None):
    save_dir = ARTIFACT_DIR / name
    save_dir.mkdir(parents=True, exist_ok=True)
    model = build_model(name).to(device)
    crit = nn.CrossEntropyLoss(label_smoothing=CFG.label_smoothing)
    trainable = [p for p in model.parameters() if p.requires_grad]
    lr = head_lr if head_lr is not None else CFG.learning_rate
    opt = torch.optim.AdamW(trainable, lr=lr, weight_decay=CFG.weight_decay)
    total_steps = CFG.epochs * len(train_loader)
    sched = warmup_cosine(opt, CFG.warmup_epochs * len(train_loader), total_steps)
    scaler = torch.amp.GradScaler('cuda', enabled=device.type == 'cuda')
    ema = EMA(model, decay=CFG.ema_decay)
    best_f1, best_state, history = -1.0, None, []
    rng = random.Random(SEED + 41)
    ema_model = build_model(name).to(device)
    for epoch in range(1, CFG.epochs + 1):
        model.train()
        run_loss, run_n = 0.0, 0
        for x, y in train_loader:
            x = x.to(device, non_blocking=True); y = y.to(device, non_blocking=True
            r = rng.random()
            if r < CFG.mix_prob / 2:
                xm, ya, yb, lam = mixup(x, y, CFG.mixup_alpha); mixed = True
            elif r < CFG.mix_prob:
                xm, ya, yb, lam = cutmix(x, y, CFG.cutmix_alpha); mixed = True
            else:
                xm, ya, yb, lam, mixed = x, y, y, 1.0, False
            opt.zero_grad(set_to_none=True)
            with torch.autocast(device_type=device.type, enabled=device.type == 'cu
                logits = model(xm)
                loss = mix_loss(crit, logits, ya, yb, lam) if mixed else crit(logit
            scaler.scale(loss).backward(); scaler.step(opt); scaler.update(); sched
            ema.update(model)
            run_loss += loss.item() * x.size(0); run_n += x.size(0)
        train_loss = run_loss / max(1, run_n)
        ema.apply_to(ema_model)
        vl, vy = collect_logits(ema_model, val_loader)
        vm = evaluate_logits(vl, vy)
```

```
        row = {'epoch': epoch, 'train_loss': round(train_loss, 4),
               'val_acc': round(vm['accuracy'], 4), 'val_f1': round(vm['macro_f1'],
               'lr': round(sched.get_last_lr()[0], 6)}
        history.append(row); print(json.dumps(row))
        torch.save({'state_dict': ema_model.state_dict(), 'epoch': epoch}, save_dir
        if vm['macro_f1'] > best_f1:
            best_f1 = vm['macro_f1']
            best_state = {k: v.detach().cpu().clone() for k, v in ema_model.state_d
            torch.save({'state_dict': best_state, 'epoch': epoch, 'val_f1': best_f1
    model.load_state_dict(best_state)
    return model, history, best_f1
```

Train DINOv2 (linear probe with higher head LR)

In [24]:
```
dino_model, dino_history, dino_best = train('dinov2_vits14', head_lr=CFG.dinov2_hea
print(f'DINOv2 best val_f1 = {dino_best:.4f}')
```

```
Using cache found in /home/spant/.cache/torch/hub/facebookresearch_dinov2_main
Using cache found in /home/spant/.cache/torch/hub/facebookresearch_dinov2_main
```

{"epoch": 1, "train_loss": 2.2215, "val_acc": 0.9112, "val_f1": 0.887, "lr": 0.00033
4}
{"epoch": 2, "train_loss": 1.6004, "val_acc": 0.9481, "val_f1": 0.935, "lr": 0.00066
7}
{"epoch": 3, "train_loss": 1.5423, "val_acc": 0.9626, "val_f1": 0.9541, "lr": 0.001}
{"epoch": 4, "train_loss": 1.5169, "val_acc": 0.9674, "val_f1": 0.9586, "lr": 0.0009
95}
{"epoch": 5, "train_loss": 1.4729, "val_acc": 0.9719, "val_f1": 0.9651, "lr": 0.0009
8}
{"epoch": 6, "train_loss": 1.4248, "val_acc": 0.9741, "val_f1": 0.9665, "lr": 0.0009
55}
{"epoch": 7, "train_loss": 1.4369, "val_acc": 0.9741, "val_f1": 0.9666, "lr": 0.0009
21}
{"epoch": 8, "train_loss": 1.4122, "val_acc": 0.9749, "val_f1": 0.9662, "lr": 0.0008
78}
{"epoch": 9, "train_loss": 1.3836, "val_acc": 0.9753, "val_f1": 0.9673, "lr": 0.0008
27}
{"epoch": 10, "train_loss": 1.404, "val_acc": 0.9758, "val_f1": 0.9685, "lr": 0.0007
7}
{"epoch": 11, "train_loss": 1.3744, "val_acc": 0.9762, "val_f1": 0.9688, "lr": 0.000
708}
{"epoch": 12, "train_loss": 1.3532, "val_acc": 0.9753, "val_f1": 0.9671, "lr": 0.000
641}
{"epoch": 13, "train_loss": 1.3595, "val_acc": 0.9755, "val_f1": 0.9675, "lr": 0.000
571}
{"epoch": 14, "train_loss": 1.3493, "val_acc": 0.9765, "val_f1": 0.9697, "lr": 0.000
5}
{"epoch": 15, "train_loss": 1.3368, "val_acc": 0.9764, "val_f1": 0.9691, "lr": 0.000
429}
{"epoch": 16, "train_loss": 1.3403, "val_acc": 0.9769, "val_f1": 0.97, "lr": 0.00035
9}
{"epoch": 17, "train_loss": 1.3399, "val_acc": 0.9769, "val_f1": 0.9701, "lr": 0.000
292}
{"epoch": 18, "train_loss": 1.3162, "val_acc": 0.9776, "val_f1": 0.9705, "lr": 0.000
23}
{"epoch": 19, "train_loss": 1.3184, "val_acc": 0.9778, "val_f1": 0.97, "lr": 0.00017
3}
{"epoch": 20, "train_loss": 1.33, "val_acc": 0.9781, "val_f1": 0.9705, "lr": 0.00012
2}
{"epoch": 21, "train_loss": 1.3075, "val_acc": 0.9778, "val_f1": 0.9702, "lr": 7.9e-
05}
{"epoch": 22, "train_loss": 1.3171, "val_acc": 0.9778, "val_f1": 0.9705, "lr": 4.5e-
05}
{"epoch": 23, "train_loss": 1.2969, "val_acc": 0.9778, "val_f1": 0.9705, "lr": 2e-0
5}
{"epoch": 24, "train_loss": 1.3192, "val_acc": 0.9778, "val_f1": 0.9705, "lr": 1e-0
5}
{"epoch": 25, "train_loss": 1.2797, "val_acc": 0.9783, "val_f1": 0.9712, "lr": 1e-0
5}
DINOv2 best val_f1 = 0.9712

Train EfficientNetV2-S (full fine-tune)

In [25]:
```
effnet_model, effnet_history, effnet_best = train('efficientnet_v2_s')
```

```
print(f'EfficientNetV2-S best val_f1 = {effnet_best:.4f}')
```

```
{"epoch": 1, "train_loss": 1.8493, "val_acc": 0.9645, "val_f1": 0.9547, "lr": 0.0001
67}
{"epoch": 2, "train_loss": 1.1793, "val_acc": 0.9797, "val_f1": 0.975, "lr": 0.00033
3}
{"epoch": 3, "train_loss": 1.1824, "val_acc": 0.9801, "val_f1": 0.9741, "lr": 0.000
5}
{"epoch": 4, "train_loss": 1.1657, "val_acc": 0.9831, "val_f1": 0.978, "lr": 0.00049
7}
{"epoch": 5, "train_loss": 1.1341, "val_acc": 0.9808, "val_f1": 0.9761, "lr": 0.0004
9}
{"epoch": 6, "train_loss": 1.0964, "val_acc": 0.9809, "val_f1": 0.9763, "lr": 0.0004
77}
{"epoch": 7, "train_loss": 1.0833, "val_acc": 0.9824, "val_f1": 0.9773, "lr": 0.0004
6}
{"epoch": 8, "train_loss": 1.0485, "val_acc": 0.9822, "val_f1": 0.9772, "lr": 0.0004
39}
{"epoch": 9, "train_loss": 1.0372, "val_acc": 0.9846, "val_f1": 0.9788, "lr": 0.0004
14}
{"epoch": 10, "train_loss": 1.0335, "val_acc": 0.9836, "val_f1": 0.9784, "lr": 0.000
385}
{"epoch": 11, "train_loss": 1.022, "val_acc": 0.9843, "val_f1": 0.9787, "lr": 0.0003
54}
{"epoch": 12, "train_loss": 0.9968, "val_acc": 0.9841, "val_f1": 0.9787, "lr": 0.000
32}
{"epoch": 13, "train_loss": 0.9824, "val_acc": 0.9834, "val_f1": 0.9778, "lr": 0.000
286}
{"epoch": 14, "train_loss": 0.9792, "val_acc": 0.9857, "val_f1": 0.9808, "lr": 0.000
25}
{"epoch": 15, "train_loss": 0.961, "val_acc": 0.9841, "val_f1": 0.9779, "lr": 0.0002
14}
{"epoch": 16, "train_loss": 0.9572, "val_acc": 0.9859, "val_f1": 0.981, "lr": 0.0001
8}
{"epoch": 17, "train_loss": 0.9403, "val_acc": 0.9855, "val_f1": 0.9811, "lr": 0.000
146}
{"epoch": 18, "train_loss": 0.9316, "val_acc": 0.9855, "val_f1": 0.9805, "lr": 0.000
115}
{"epoch": 19, "train_loss": 0.9191, "val_acc": 0.9852, "val_f1": 0.9799, "lr": 8.6e-
05}
{"epoch": 20, "train_loss": 0.9409, "val_acc": 0.9846, "val_f1": 0.9792, "lr": 6.1e-
05}
{"epoch": 21, "train_loss": 0.9237, "val_acc": 0.9838, "val_f1": 0.9779, "lr": 4e-0
5}
{"epoch": 22, "train_loss": 0.9344, "val_acc": 0.9841, "val_f1": 0.9784, "lr": 2.3e-
05}
{"epoch": 23, "train_loss": 0.9045, "val_acc": 0.9832, "val_f1": 0.9773, "lr": 1e-0
5}
{"epoch": 24, "train_loss": 0.9178, "val_acc": 0.9841, "val_f1": 0.9791, "lr": 5e-0
6}
{"epoch": 25, "train_loss": 0.8947, "val_acc": 0.9839, "val_f1": 0.9786, "lr": 5e-0
6}
EfficientNetV2-S best val_f1 = 0.9811
```

Calibration on real-world held-out (target precision 0.80)

In [26]:

```
def fit_temperature(logits, labels):
    lo, la = logits.to(device), labels.to(device)
    t = torch.ones(1, device=device, requires_grad=True)
    opt = torch.optim.LBFGS([t], lr=0.01, max_iter=50)
    crit = nn.CrossEntropyLoss()
    def closure():
        opt.zero_grad()
        loss = crit(lo / t.clamp(0.5, 5.0), la)
        loss.backward()
        return loss
    opt.step(closure)
    return float(t.detach().clamp(0.5, 5.0).item())

def uncertainty(logits, T):
    p = torch.softmax(logits / T, dim=1)
    top2 = p.topk(2, dim=1).values
    return (p.argmax(1), top2[:, 0], top2[:, 0] - top2[:, 1],
            -(p * torch.log(p.clamp_min(1e-8))).sum(1) / math.log(NUM_CLASSES))

def search_thresholds(logits, labels, T, target):
    pred, mp, mg, en = uncertainty(logits, T)
    correct = pred == labels
    best = None
    for c in np.linspace(0.35, 0.95, 13):
        for m in np.linspace(0.05, 0.45, 9):
            for e in np.linspace(0.25, 0.85, 13):
                mask = (mp >= c) & (mg >= m) & (en <= e)
                if mask.sum() < 20:
                    continue
                prec = float(correct[mask].float().mean())
                cov = float(mask.float().mean())
                if prec >= target and (best is None or cov > best['confident_covera
                    best = {'max_prob_min': round(float(c), 4), 'margin_min': round
                            'entropy_max': round(float(e), 4),
                            'confident_precision': round(prec, 4), 'confident_cover
    return best or {'max_prob_min': 0.60, 'margin_min': 0.15, 'entropy_max': 0.65,
                    'confident_precision': None, 'confident_coverage': None}

def calibrate(model, name):
    vl, vy = collect_logits(model, val_loader)
    cl, cy = collect_logits(model, calibration_loader)
    T = fit_temperature(vl, vy)
    thr = search_thresholds(cl, cy, T, CFG.target_confident_precision)
    print(f'{name}: T={T:.3f}  thresholds={thr}')
    return T, thr

dino_T, dino_thr = calibrate(dino_model, 'dinov2_vits14')
effnet_T, effnet_thr = calibrate(effnet_model, 'efficientnet_v2_s')
```

```
dinov2_vits14: T=0.849  thresholds={'max_prob_min': 0.35, 'margin_min': 0.4, 'entrop
y_max': 0.45, 'confident_precision': 0.8012, 'confident_coverage': 0.644}
efficientnet_v2_s: T=0.874  thresholds={'max_prob_min': 0.85, 'margin_min': 0.45, 'e
ntropy_max': 0.85, 'confident_precision': 0.8013, 'confident_coverage': 0.624}
```

Per-source evaluation (single-model)

In [27]:
```python
def evaluate_all(model, T, name):
    print(f'\n=== {name} (T={T:.3f}) ===')
    out = {}
    for src, ldr in test_loaders.items():
        lg, lb = collect_logits(model, ldr)
        m = evaluate_logits(lg, lb, T)
        m['n'] = int(lb.size(0))
        out[src] = m
        print(f'  {src:25s} n={m["n"]:>5d}  acc={m["accuracy"]:.4f}  f1={m["macro_f
    return out

dino_per_source = evaluate_all(dino_model, dino_T, 'dinov2_vits14')
effnet_per_source = evaluate_all(effnet_model, effnet_T, 'efficientnet_v2_s')
```

```
=== dinov2_vits14 (T=0.849) ===
  plantvillage_test         n= 5417  acc=0.9904  f1=0.9879
  plantdoc_test             n=  250  acc=0.6520  f1=0.5615

=== efficientnet_v2_s (T=0.874) ===
  plantvillage_test         n= 5417  acc=0.9974  f1=0.9964
  plantdoc_test             n=  250  acc=0.6720  f1=0.6537
```

Ensemble evaluation (DINOv2 + EffNet, averaged softmax)

In [28]:
```python
@torch.no_grad()
def collect_softmax(model, loader, T):
    model.eval()
    P, Y = [], []
    for x, y in loader:
        logits = model(x.to(device, non_blocking=True))
        P.append(torch.softmax(logits / T, dim=1).cpu())
        Y.append(y)
    return torch.cat(P), torch.cat(Y)

def evaluate_ensemble(loader):
    pd_, yd = collect_softmax(dino_model, loader, dino_T)
    pe_, ye = collect_softmax(effnet_model, loader, effnet_T)
    assert torch.equal(yd, ye)
    probs = (pd_ + pe_) / 2.0
    pred = probs.argmax(1)
    return {
        'accuracy': float((pred == yd).float().mean()),
        'macro_f1': float(f1_score(yd.numpy(), pred.numpy(), average='macro', zero_
        'n': int(yd.size(0)),
    }
```

```
print(f'\n=== ENSEMBLE (DINOv2 + EffNet, averaged) ===')
ensemble_per_source = {}
for src, ldr in test_loaders.items():
    m = evaluate_ensemble(ldr)
    ensemble_per_source[src] = m
    print(f'  {src:25s} n={m["n"]:>5d}  acc={m["accuracy"]:.4f}  f1={m["macro_f1"]:
```

```
=== ENSEMBLE (DINOv2 + EffNet, averaged) ===
  plantvillage_test         n= 5417  acc=0.9974  f1=0.9964
  plantdoc_test             n=  250  acc=0.6840  f1=0.6641
```

TTA + ensemble (final number)

In [29]:

```
tta_base = transforms.Compose([
    transforms.Resize(CFG.image_size_eval + 32), transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])

def five_crop_flips(t, sz):
    _, h, w = t.shape
    crops = []
    cy, cx = (h - sz) // 2, (w - sz) // 2
    for (y, x) in [(0, 0), (0, w - sz), (h - sz, 0), (h - sz, w - sz), (cy, cx)]:
        c = t[:, y:y+sz, x:x+sz]
        crops.append(c)
        crops.append(torch.flip(c, dims=[2]))
    return torch.stack(crops)

@torch.no_grad()
def predict_tta(model, image_path, T, crop_size=CFG.image_size_eval):
    img = Image.open(image_path).convert('RGB')
    crops = five_crop_flips(tta_base(img), crop_size).to(device)
    model.eval()
    return torch.softmax(model(crops) / T, dim=1).mean(0).cpu()

@torch.no_grad()
def predict_tta_ensemble(image_path):
    pd_ = predict_tta(dino_model, image_path, dino_T, CFG.image_size_eval)
    pe_ = predict_tta(effnet_model, image_path, effnet_T, CFG.image_size_eval)
    return (pd_ + pe_) / 2.0

def evaluate_tta_ensemble(samples, max_n=400):
    rng = random.Random(SEED + 71)
    sub = samples if len(samples) <= max_n else rng.sample(samples, max_n)
    correct = sum(1 for s in sub if int(predict_tta_ensemble(s.path).argmax()) == s
    return correct / max(1, len(sub)), len(sub)

print('\n--- TTA + ENSEMBLE ---')
tta_ensemble_per_source = {}
for tname, ts in test_sets.items():
    if ts:
        acc, n = evaluate_tta_ensemble(ts)
        tta_ensemble_per_source[tname] = {'accuracy': acc, 'n': n}
        print(f'  {tname:25s} TTA+ensemble acc={acc:.4f} (n={n})')
```

```
--- TTA + ENSEMBLE ---
  plantvillage_test         TTA+ensemble acc=1.0000 (n=400)
  plantdoc_test             TTA+ensemble acc=0.7160 (n=250)
```

Save bundles + report

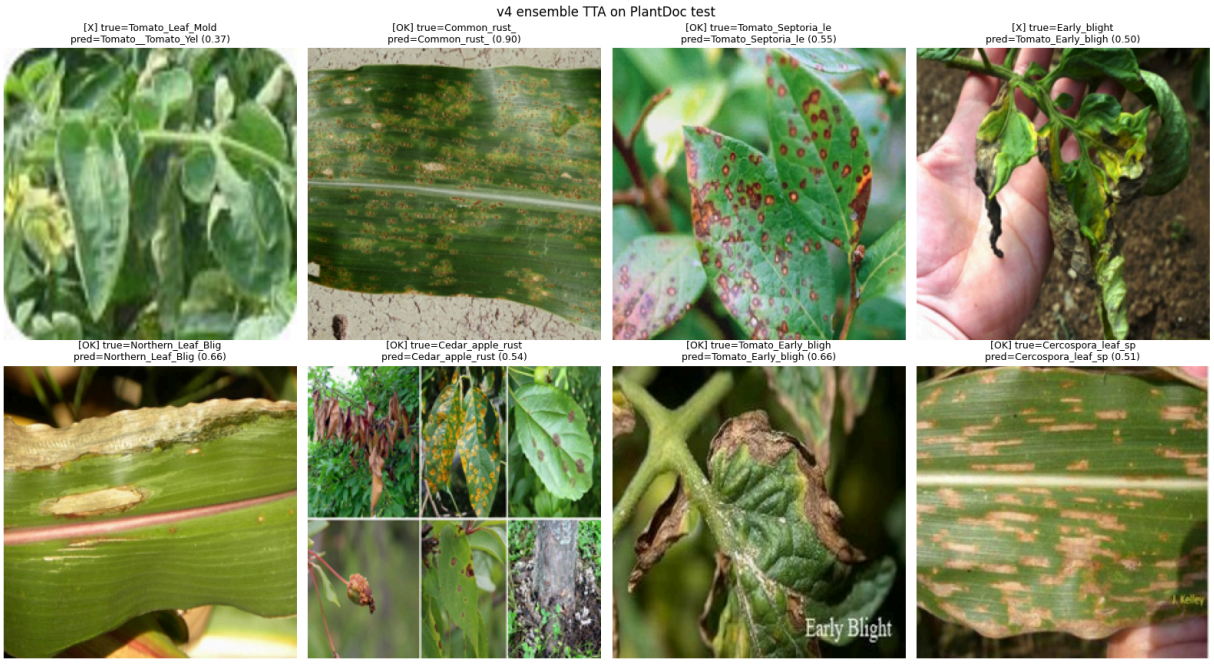
In [30]:
```python
def save_bundle(name, model, history, T, thr, per_source):
    w = ARTIFACT_DIR / f'{name}_cropscan_v4.pth'
    b = ARTIFACT_DIR / f'{name}_cropscan_v4_bundle.pt'
    torch.save(model.state_dict(), w)
    torch.save({
        'state_dict': model.state_dict(),
        'temperature': T,
        'thresholds': thr,
        'class_names': CLASS_NAMES,
        'version': 'cropscan-v4',
        'architecture': name,
        'history': history,
        'per_source_test_metrics': per_source,
    }, b)
    print(f'  {w}')
    print(f'  {b}')

save_bundle('dinov2_vits14',     dino_model,    dino_history,    dino_T,    dino_th
save_bundle('efficientnet_v2_s', effnet_model,  effnet_history,  effnet_T,  effnet_

report = {
    'class_names': CLASS_NAMES,
    'data_sizes': {
        'train_total': len(train_samples),
        'pv_train': len(pv_train), 'synthetic': len(synth_samples), 'pd_train': len
        'val_total': len(val_samples), 'calibration_total': len(calibration_samples
    },
    'test_sizes': {k: len(v) for k, v in test_sets.items()},
    'dinov2_vits14':      {'history': dino_history,   'temperature': dino_T,   'thr
    'efficientnet_v2_s':  {'history': effnet_history, 'temperature': effnet_T, 'thr
    'ensemble':           {'per_source': ensemble_per_source},
    'ensemble_tta':       {'per_source': tta_ensemble_per_source},
}
json.dump(report, open(ARTIFACT_DIR / 'training_report.json', 'w'), indent=2)
json.dump(CLASS_NAMES, open(ARTIFACT_DIR / 'labels.json', 'w'), indent=2)
print(f'\nReport: {ARTIFACT_DIR / "training_report.json"}')
```

```
  artifacts/v4/dinov2_vits14_cropscan_v4.pth
  artifacts/v4/dinov2_vits14_cropscan_v4_bundle.pt
  artifacts/v4/efficientnet_v2_s_cropscan_v4.pth
  artifacts/v4/efficientnet_v2_s_cropscan_v4_bundle.pt

Report: artifacts/v4/training_report.json
```

Visualize ensemble TTA predictions on PlantDoc

In [31]:
```python
def visualize_ensemble(samples, n=8):
    if not samples:
```

```
        return
    rng = random.Random(SEED + 101)
    picks = rng.sample(samples, min(n, len(samples)))
    fig, axes = plt.subplots(2, 4, figsize=(16, 9))
    for ax, s in zip(axes.flat, picks):
        probs = predict_tta_ensemble(s.path)
        top = int(probs.argmax())
        ax.imshow(Image.open(s.path).convert('RGB').resize((224, 224)))
        ax.axis('off')
        mark = 'OK' if top == s.label else 'X'
        true = CLASS_NAMES[s.label].split('___')[-1][:18]
        pred = CLASS_NAMES[top].split('___')[-1][:18]
        ax.set_title(f'[{mark}] true={true}\npred={pred} ({probs[top]:.2f})', fonts
    plt.suptitle('v4 ensemble TTA on PlantDoc test')
    plt.tight_layout()
    plt.savefig(ARTIFACT_DIR / 'preview_ensemble.png', dpi=100, bbox_inches='tight'
    plt.show()

visualize_ensemble(pd_test)
```



Backend integration

v4 introduces a Vision Transformer (DINOv2) alongside EfficientNetV2-S, plus an ensemble step at inference. Required `backend/app/inference.py` changes:

1. Copy artifacts to `backend/models/`:
   - `dinov2_vits14_cropscan_v4.pth`
   - `efficientnet_v2_s_cropscan_v4.pth`
   - matching `*_bundle.pt`
   - `training_report.json`, `labels.json`
2. Add the DINOv2 builder (mirror the notebook):

```
import torch.nn as nn

class DINOv2Classifier(nn.Module):
    def __init__(self, num_classes, embed_dim=384, hidden=512,
dropout=0.2):
        super().__init__()
        self.backbone = torch.hub.load('facebookresearch/dinov2',
'dinov2_vits14', skip_validation=True)
        for p in self.backbone.parameters():
            p.requires_grad = False
        self.backbone.eval()
        self.head = nn.Sequential(
            nn.LayerNorm(embed_dim),
            nn.Linear(embed_dim, hidden),
            nn.GELU(),
            nn.Dropout(dropout),
            nn.Linear(hidden, num_classes),
        )
    def train(self, mode=True):
        super().train(mode)
        self.backbone.eval()
        return self
    def forward(self, x):
        return self.head(self.backbone(x))

def _build_dinov2_vits14():
    return DINOv2Classifier(len(CLASS_NAMES))
```

3. At inference, average softmax from both models (each divided by its own temperature). Wire the abstention thresholds from `training_report.json` into the existing logic — note v4 targets 80% confident-precision, not 90%.

4. Eval resolution is 280, not 224. Make sure the inference transform uses `Resize(312) + CenterCrop(280)`.

In [ ]:

In [ ]: